COMP 3920 Database Systems 1

MySQL Transactions

Introduction:
The goal of this document is to serve as a walkthrough working with Transactions and the different isolation levels in MySQL.

For this document, we will be using command prompt commands and MySQL Workbench in MySQL (in either Windows or Linux).
In general there should be little difference in the commands used in this document between Windows and Linux.

Please follow along **from within a VirtualBox Virtual Machine (VM)**.

We are going to be create a number of scenarios to test how transactions work:

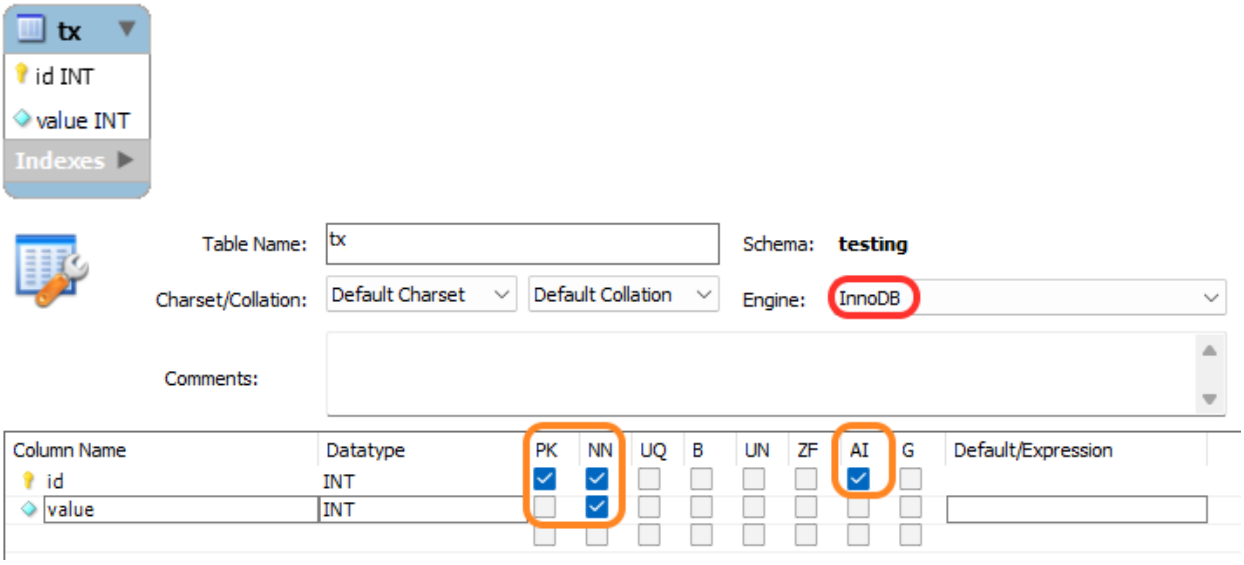
A. Transactions and Rollbacks
B. Transactions and Commits
C. Atomic Transactions
D. Ended Transactions
E. Isolated Consistent Backups
F. MyISAM is Not Transaction Safe
G. MyISAM Read Locks Block Writers
H. Deadlock
I. Implicit Commit with DDL
J. Uncommitted Transactions Prevent Other's DDL
K. Isolation Level – Read Uncommitted
L. Isolation Level – Serializable

IMPORTANT!

For each scenario complete the Predictions Section **BEFORE** running the commands and capturing your screenshots. The Predictions Sheet needs to be submitted into the Learning Hub.

Step 1:
Create a new database called `testing`.

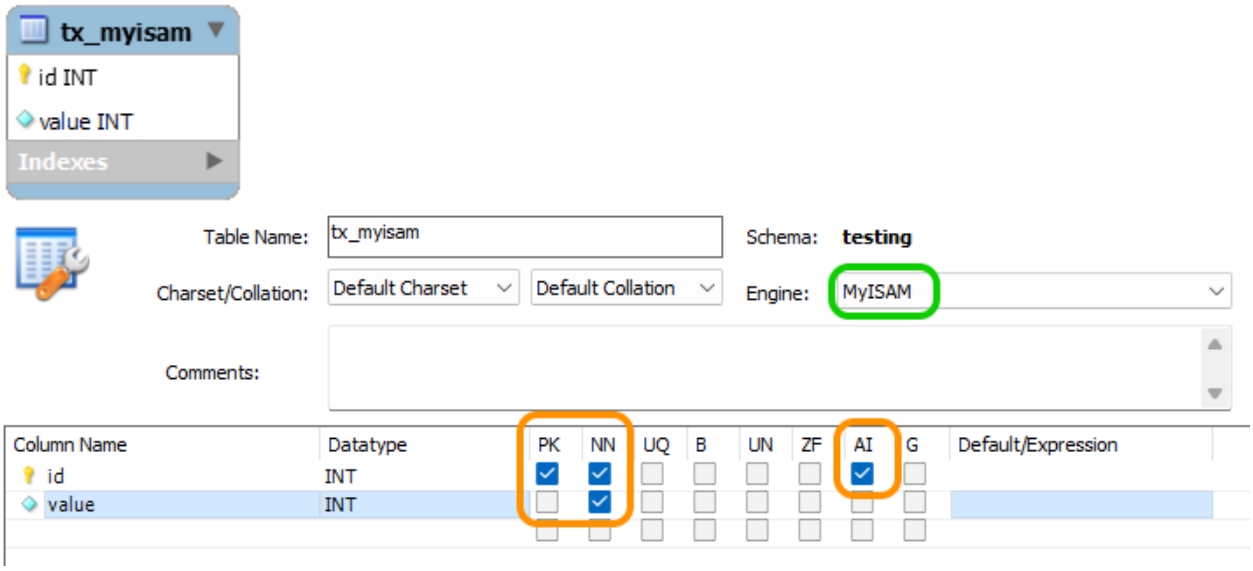
Create a table called `tx` as such:



IMPORTANT!

Ensure the `id` column is marked as `PRIMARY_KEY` AND `AUTO_INCREMENT`.

Create another table called `tx_myisam` ***that uses table storage engine MyISAM*** as such:



IMPORTANT!

Use engine **MyISAM** for table `tx_myisam`. (Use the default engine InnoDB for table `tx`).

Insert into your new table `tx` the following rows:

```
24 •  select * from tx;
```

Result Grid | Filter Rows: | Ed

id	value
1	1000
2	2000
3	3000
4	4000

Insert into your new table `tx_myisam` the following rows:

```
1 •  select * from testing.tx_myisam;
```

Result Grid | Filter Rows: | Edit:

id	value
1	10
2	20
3	30
4	40
NULL	NULL

Confirm that the `tx` table is of type InnoDB by running a `SHOW CREATE TABLE` command:

```
21 •  SHOW CREATE TABLE tx;
22
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Table	Create Table
tx	CREATE TABLE `t

Open Value in Viewer
Set Field to NULL
Mark Field Value as a Function/Literal
Delete Row(s)

Right click on the resultset output and click "Open Value in Viewer".

Switch to the "Text" tab at the top, if needed.

Binary Text

```
1  CREATE TABLE `tx` (
2    `id` int NOT NULL AUTO_INCREMENT,
3    `value` int NOT NULL,
4    PRIMARY KEY (`id`)
5  ) ENGINE=InnoDB AUTO_INCREMENT=14 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
```

Note the `Engine=InnoDB`, which indicates it is using the InnoDB table storage engine.

Confirm with a `SHOW CREATE TABLE` command that `tx_myisam` is of has a MyISAM table storage engine.

Binary Text

```
1  CREATE TABLE `tx_myisam` (
2    `id` int NOT NULL AUTO_INCREMENT,
3    `value` int NOT NULL,
4    PRIMARY KEY (`id`)
5  ) ENGINE=MyISAM DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
```

Note the `Engine=MyISAM`, which indicates it is using the MyISAM table storage engine.

Step 2:

Scenario A: Transactions and Rollbacks

Open 2 terminal windows and login to MySQL on both, using `root`.

Set your default database to `testing`.

Each of the terminal windows is indicated below in the blue boxes with the white text (one on the left and the other on the right). Execute the commands in the order shown to the left of each terminal window.

This is our first example of a transaction.

Terminal Window #1

```
1  START TRANSACTION;
2  SELECT tx.*, NOW() FROM tx;

4  UPDATE tx SET value = 1234
   WHERE id = 1;
5  SELECT tx.*, NOW() FROM tx;

7  ROLLBACK;
8  SELECT tx.*, NOW() FROM tx;
```

Terminal Window #2

```
3  SELECT tx.*, NOW() FROM tx;

6  SELECT tx.*, NOW() FROM tx;

9  SELECT tx.*, NOW() FROM tx;
```

For example:
Start with #1 in Terminal Window #1 with `START TRANSACTION;`
Next #2 in Terminal Window #1 with `SELECT * FROM tx;`
Then for #3 in Terminal Window #2 execute `SELECT * FROM tx;`
…
#8 in Terminal Window #1 with `SELECT * FROM tx;`
Up to #9 in Terminal Window #2 with the last `SELECT * FROM tx;`

<u>Predictions Section:</u>

What is the value of `value` for `id=1` in Step 5?
What is the value of `value` for `id=1` in Step 6? Is this the same as in Step 5?
What is the value of `value` for `id=1` in Step 8?
What is the value of `value` for `id=1` in Step 9? Is this the same as in Step 8?

You should notice 2 things with Scenario A: Transaction and Rollbacks:

1. Terminal Window #1 has a transaction and changes in this transaction will not affect any other users (for example Terminal Window #2).
2. A rollback of the transaction in Terminal Window #1 reverses all the changes made up to that point in the transaction.

Submit your screenshot of steps 1-6 (before the rollback):

Submit

01_Scenario_A_before_rollback.jpg

```
mysql> start transaction;
Query OK, 0 rows affected (0.00 sec)

mysql> select tx.*, now() from tx;
+----+-------+---------------------+
| id | value | now()               |
+----+-------+---------------------+
|  1 |  1000 | [illegible]         |
|  2 |  2000 | [illegible]         |
|  3 |  3000 | [illegible]         |
|  4 |  4000 | [illegible]         |
+----+-------+---------------------+
4 rows in set (0.00 sec)

mysql> update tx set value = 1234 where id = 1;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> select tx.*, now() from tx;
+----+-------+---------------------+
| id | value | now()               |
+----+-------+---------------------+
|  1 |  1234 | [illegible]         |
|  2 |  2000 | [illegible]         |
|  3 |  3000 | [illegible]         |
|  4 |  4000 | [illegible]         |
+----+-------+---------------------+
4 rows in set (0.00 sec)

mysql>
```

```
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql> select tx.*, now() from tx;
+----+-------+---------------------+
| id | value | now()               |
+----+-------+---------------------+
|  1 |  1000 | [illegible]         |
|  2 |  2000 | [illegible]         |
|  3 |  3000 | [illegible]         |
|  4 |  4000 | [illegible]         |
+----+-------+---------------------+
4 rows in set (0.00 sec)

mysql> select tx.*, now() from tx;
+----+-------+---------------------+
| id | value | now()               |
+----+-------+---------------------+
|  1 |  1000 | [illegible]         |
|  2 |  2000 | [illegible]         |
|  3 |  3000 | [illegible]         |
|  4 |  4000 | [illegible]         |
+----+-------+---------------------+
4 rows in set (0.00 sec)

mysql>
```

Submit your screenshot of steps 1-9 (after the rollback):

Submit

02_Scenario_A_after_rollback.jpg

```
mysql> update tx set value = 1234 where id = 1;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> select tx.*, now() from tx;
+----+-------+---------------------+
| id | value | now()               |
+----+-------+---------------------+
|  1 |  1234 | [illegible]         |
|  2 |  2000 | [illegible]         |
|  3 |  3000 | [illegible]         |
|  4 |  4000 | [illegible]         |
+----+-------+---------------------+
4 rows in set (0.00 sec)

mysql> rollback;
Query OK, 0 rows affected (0.00 sec)

mysql> select tx.*, now() from tx;
+----+-------+---------------------+
| id | value | now()               |
+----+-------+---------------------+
|  1 |  1000 | [illegible]         |
|  2 |  2000 | [illegible]         |
|  3 |  3000 | [illegible]         |
|  4 |  4000 | [illegible]         |
+----+-------+---------------------+
4 rows in set (0.00 sec)

mysql>
```

```
|  1 |  1000 | [illegible]         |
|  2 |  2000 | [illegible]         |
|  3 |  3000 | [illegible]         |
|  4 |  4000 | [illegible]         |
+----+-------+---------------------+
4 rows in set (0.00 sec)

mysql> select tx.*, now() from tx;
+----+-------+---------------------+
| id | value | now()               |
+----+-------+---------------------+
|  1 |  1000 | [illegible]         |
|  2 |  2000 | [illegible]         |
|  3 |  3000 | [illegible]         |
|  4 |  4000 | [illegible]         |
+----+-------+---------------------+
4 rows in set (0.00 sec)

mysql> select tx.*, now() from tx;
+----+-------+---------------------+
| id | value | now()               |
+----+-------+---------------------+
|  1 |  1000 | [illegible]         |
|  2 |  2000 | [illegible]         |
|  3 |  3000 | [illegible]         |
|  4 |  4000 | [illegible]         |
+----+-------+---------------------+
4 rows in set (0.00 sec)

mysql>
```

Step 3:

Scenario B: Transactions and Commits

Keep the 2 terminal windows logged in to MySQL as `root`.

Run this command to reset the data back to the initial values (if needed):
`UPDATE tx SET value = id * 1000;`

IMPORTANT: Scroll the window up to the top so your screenshot only shows the current scenario.

Run the following commands on their appropriate Terminal Windows in sequence.

This scenario is very similar to Scenario A, except the `ROLLBACK;` is replaced by `COMMIT;`.

Terminal Window #1

```
1  START TRANSACTION;
2  SELECT tx.*, NOW() FROM tx;

4  UPDATE tx SET value = 1557
   WHERE id = 1;
5  SELECT tx.*, NOW() FROM tx;

7  COMMIT;
8  SELECT tx.*, NOW() FROM tx;
```

Terminal Window #2

```
3  SELECT tx.*, NOW() FROM tx;

6  SELECT tx.*, NOW() FROM tx;

9  SELECT tx.*, NOW() FROM tx;
```

<u>Predictions Section:</u>

What is the value of `value` for `id=1` in Step 5?
What is the value of `value` for `id=1` in Step 6? Is this the same as in Step 5?
What is the value of `value` for `id=1` in Step 8?
What is the value of `value` for `id=1` in Step 9? Is this the same as in Step 8?

You should notice 2 things with Scenario B: Transaction and Commits:

1. Terminal Window #1 has a transaction and changes in this transaction will not affect any other users (for example Terminal Window #2) – just like Scenario A.
2. A commit of the transaction in Terminal Window #1 finalizes all the changes made up to that point in the transaction and will make the changes visible to all other users (including Terminal Window #2).

Submit your screenshot of steps 1-6 (before the commit):

03_Scenario_B_before_commit.jpg

Submit

```
mysql> start transaction;
Query OK, 0 rows affected (0.00 sec)

mysql> select tx.*, now() from tx;
+----+-------+---------------------+
| id | value | now()               |
+----+-------+---------------------+
|  1 |  1000 | [illegible]         |
|  2 |  2000 | [illegible]         |
|  3 |  3000 | [illegible]         |
|  4 |  4000 | [illegible]         |
+----+-------+---------------------+
4 rows in set (0.00 sec)

mysql> update tx set value = 1557 where id = 1;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> select tx.*, now() from tx;
+----+-------+---------------------+
| id | value | now()               |
+----+-------+---------------------+
|  1 |  1557 | [illegible]         |
|  2 |  2000 | [illegible]         |
|  3 |  3000 | [illegible]         |
|  4 |  4000 | [illegible]         |
+----+-------+---------------------+
4 rows in set (0.00 sec)

mysql>
```

```
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql> select tx.*, now() from tx;
+----+-------+---------------------+
| id | value | now()               |
+----+-------+---------------------+
|  1 |  1000 | [illegible]         |
|  2 |  2000 | [illegible]         |
|  3 |  3000 | [illegible]         |
|  4 |  4000 | [illegible]         |
+----+-------+---------------------+
4 rows in set (0.00 sec)

mysql> select tx.*, now() from tx;
+----+-------+---------------------+
| id | value | now()               |
+----+-------+---------------------+
|  1 |  1000 | [illegible]         |
|  2 |  2000 | [illegible]         |
|  3 |  3000 | [illegible]         |
|  4 |  4000 | [illegible]         |
+----+-------+---------------------+
4 rows in set (0.00 sec)

mysql>
```

Submit your screenshot of steps 1-9 (after the commit):

04_Scenario_B_after_commit.jpg

Submit

```
mysql> update tx set value = 1557 where id = 1;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> select tx.*, now() from tx;
+----+-------+---------------------+
| id | value | now()               |
+----+-------+---------------------+
|  1 |  1557 | [illegible]         |
|  2 |  2000 | [illegible]         |
|  3 |  3000 | [illegible]         |
|  4 |  4000 | [illegible]         |
+----+-------+---------------------+
4 rows in set (0.00 sec)

mysql> commit;
Query OK, 0 rows affected (0.01 sec)

mysql> select tx.*, now() from tx;
+----+-------+---------------------+
| id | value | now()               |
+----+-------+---------------------+
|  1 |  1557 | [illegible]         |
|  2 |  2000 | [illegible]         |
|  3 |  3000 | [illegible]         |
|  4 |  4000 | [illegible]         |
+----+-------+---------------------+
4 rows in set (0.00 sec)

mysql>
```

```
|  1 |  1000 | [illegible]         |
|  2 |  2000 | [illegible]         |
|  3 |  3000 | [illegible]         |
|  4 |  4000 | [illegible]         |
+----+-------+---------------------+
4 rows in set (0.00 sec)

mysql> select tx.*, now() from tx;
+----+-------+---------------------+
| id | value | now()               |
+----+-------+---------------------+
|  1 |  1000 | [illegible]         |
|  2 |  2000 | [illegible]         |
|  3 |  3000 | [illegible]         |
|  4 |  4000 | [illegible]         |
+----+-------+---------------------+
4 rows in set (0.00 sec)

mysql> select tx.*, now() from tx;
+----+-------+---------------------+
| id | value | now()               |
+----+-------+---------------------+
|  1 |  1557 | [illegible]         |
|  2 |  2000 | [illegible]         |
|  3 |  3000 | [illegible]         |
|  4 |  4000 | [illegible]         |
+----+-------+---------------------+
4 rows in set (0.00 sec)

mysql>
```

Step 4:

Scenario C: Atomic Transactions

Keep the 2 terminal windows logged in to MySQL as `root`.

Run this command to reset the data back to the initial values (if needed):
`UPDATE tx SET value = id * 1000;`

IMPORTANT: Scroll the window up to the top so your screenshot only shows the current scenario.

Run the following commands on their appropriate Terminal Windows in sequence.

This scenario is similar to the first scenario, except there can be multiple modifications to the table (multiple INSERTs, UPDATEs and/or DELETEs).

Terminal Window #1

```
1  START TRANSACTION;

3  UPDATE tx SET value = 2243
   WHERE id = 2;

5  DELETE FROM tx WHERE id = 3;

6  SELECT tx.*, NOW() FROM tx;
7  ROLLBACK;
8  SELECT tx.*, NOW() FROM tx;
```

Terminal Window #2

```
2  SELECT tx.*, NOW() FROM tx;

4  SELECT tx.*, NOW() FROM tx;

9  SELECT tx.*, NOW() FROM tx;
```

<u>Predictions Section:</u>

What is the value of `value` for `id=2` in Step 4?
What is the value of `value` for `id=3` in Step 4?
What is the value of `value` for `id=2` in Step 9?
What is the value of `value` for `id=3` in Step 9?

You should notice 2 things with Scenario C: Atomic Transactions:

1. Terminal Window #1 has a transaction and *partial* changes in this transaction will <u>not</u> be visible outside this transaction. (for example, Terminal Window #2 will not see the UPDATE from Terminal Window #1).

   This is useful to guarantee that both the UPDATE and DELETE happen together or not at all.

2. The changes inside the transaction in Terminal #1 would be visible to Terminal #2, if the transaction were committed. Instead, the rollback reverts both the UPDATE and the DELETE.

Submit your screenshot of steps 1-6 (before the rollback):

Submit

05_Scenario_C_before_rollback.jpg

```
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql> start transaction;
Query OK, 0 rows affected (0.00 sec)

mysql> update tx set value = 2243 where id = 2;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> delete from tx where id = 3;
Query OK, 1 row affected (0.00 sec)

mysql> select tx.*, now() from tx;
+----+-------+---------------------+
| id | value | now()               |
+----+-------+---------------------+
|  1 |  1000 |                     |
|  2 |  2243 |                     |
|  4 |  4000 |                     |
+----+-------+---------------------+
3 rows in set (0.00 sec)

mysql>
```

```
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql> select tx.*, now() from tx;
+----+-------+---------------------+
| id | value | now()               |
+----+-------+---------------------+
|  1 |  1000 |                     |
|  2 |  2000 |                     |
|  3 |  3000 |                     |
|  4 |  4000 |                     |
+----+-------+---------------------+
4 rows in set (0.00 sec)

mysql> select tx.*, now() from tx;
+----+-------+---------------------+
| id | value | now()               |
+----+-------+---------------------+
|  1 |  1000 |                     |
|  2 |  2000 |                     |
|  3 |  3000 |                     |
|  4 |  4000 |                     |
+----+-------+---------------------+
4 rows in set (0.00 sec)

mysql>
```

Submit your screenshot of steps 1-9 (after the rollback):

Submit

06_Scenario_C_after_rollback.jpg

```
Rows matched: 1  Changed: 1  Warnings: 0

mysql> delete from tx where id = 3;
Query OK, 1 row affected (0.00 sec)

mysql> select tx.*, now() from tx;
+----+-------+---------------------+
| id | value | now()               |
+----+-------+---------------------+
|  1 |  1000 |                     |
|  2 |  2243 |                     |
|  4 |  4000 |                     |
+----+-------+---------------------+
3 rows in set (0.00 sec)

mysql> rollback;
Query OK, 0 rows affected (0.00 sec)

mysql> select tx.*, now() from tx;
+----+-------+---------------------+
| id | value | now()               |
+----+-------+---------------------+
|  1 |  1000 |                     |
|  2 |  2000 |                     |
|  3 |  3000 |                     |
|  4 |  4000 |                     |
+----+-------+---------------------+
4 rows in set (0.00 sec)

mysql>
```

```
|  1 |  1000 |                     |
|  2 |  2000 |                     |
|  3 |  3000 |                     |
|  4 |  4000 |                     |
+----+-------+---------------------+
4 rows in set (0.00 sec)

mysql> select tx.*, now() from tx;
+----+-------+---------------------+
| id | value | now()               |
+----+-------+---------------------+
|  1 |  1000 |                     |
|  2 |  2000 |                     |
|  3 |  3000 |                     |
|  4 |  4000 |                     |
+----+-------+---------------------+
4 rows in set (0.00 sec)

mysql> select tx.*, now() from tx;
+----+-------+---------------------+
| id | value | now()               |
+----+-------+---------------------+
|  1 |  1000 |                     |
|  2 |  2000 |                     |
|  3 |  3000 |                     |
|  4 |  4000 |                     |
+----+-------+---------------------+
4 rows in set (0.00 sec)

mysql>
```

Step 5:

Scenario D: Ended Transactions

Keep the 2 terminal windows logged in to MySQL as `root`.

Run this command to reset the data back to the initial values (if needed):
`UPDATE tx SET value = id * 1000;`

IMPORTANT: Scroll the window up to the top so your screenshot only shows the current scenario.

Run the following commands on their appropriate Terminal Windows in sequence.

In this scenario we look at what happens if we run a ROLLBACK right after a COMMIT, does it work? What about a COMMIT after a ROLLBACK?

Run all the commands on Terminal #1, take a screenshot, then run all the commands on Terminal #2 and take another screenshot.

Terminal Window #1

```
1  START TRANSACTION;
2  UPDATE tx SET value = 3358
   WHERE id = 3;

3  SELECT tx.*, NOW() FROM tx;
4  COMMIT;
5  ROLLBACK;

6  SELECT tx.*, NOW() FROM tx;
```

Terminal Window #2

```
7  START TRANSACTION;
8  UPDATE tx SET value = 4706
   WHERE id = 4;

9  SELECT tx.*, NOW() FROM tx;
10 ROLLBACK;
11 COMMIT;

12 SELECT tx.*, NOW() FROM tx;
```

<u>Predictions Section:</u>

What is the value of `value` for `id=3` in Step 6?
What is the value of `value` for `id=4` in Step 12?

You should notice 2 things with Scenario D: Ended Transactions:

1. Running a `ROLLBACK` after a `COMMIT` does nothing. Once committed, the transaction cannot be reverted using a `ROLLBACK` – it's too late!
2. Running a `COMMIT` after a `ROLLBACK` also does nothing. Once a transaction has ended and the changes reverted by the `ROLLBACK`, they are gone and cannot be committed.

Submit your screenshot of steps 1-6 (steps on Terminal #1):

07_Scenario_D_commit_rollback.jpg

Submit

```
mysql> start transaction;
Query OK, 0 rows affected (0.00 sec)

mysql> update tx set value = 3358 where id = 3;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> select tx.*, now() from tx;
+----+-------+---------------------+
| id | value | now()               |
+----+-------+---------------------+
|  1 |  1000 |                     |
|  2 |  2000 |                     |
|  3 |  3358 |                     |
|  4 |  4000 |                     |
+----+-------+---------------------+
4 rows in set (0.00 sec)

mysql> commit;
Query OK, 0 rows affected (0.01 sec)

mysql> rollback;
Query OK, 0 rows affected (0.00 sec)

mysql> select tx.*, now() from tx;
+----+-------+---------------------+
| id | value | now()               |
+----+-------+---------------------+
|  1 |  1000 |                     |
|  2 |  2000 |                     |
|  3 |  3358 |                     |
|  4 |  4000 |                     |
+----+-------+---------------------+
4 rows in set (0.00 sec)

mysql>
```

```
mysql>
mysql>
mysql>
```

Step 6:

Scenario E: Isolated Consistent Backup

Keep the 2 terminal windows logged in to MySQL as `root`.

Run this command to reset the data back to the initial values (if needed):
`UPDATE tx SET value = id * 1000;`

IMPORTANT: Scroll the window up to the top so your screenshot only shows the current scenario.

Run the following commands on their appropriate Terminal Windows in sequence.

In this scenario we create a transaction is Terminal Window #1 and modify data in Terminal Window #2 – we'll see what effect those changes have on our transaction.

Terminal Window #1

```
1  START TRANSACTION;
2  SELECT tx.*, NOW() FROM tx;



5  SELECT tx.*, NOW() FROM tx;
6  ROLLBACK;
7  SELECT tx.*, NOW() FROM tx;
```

Terminal Window #2

```
3  UPDATE tx SET value = 4994
   WHERE id = 4;
4  SELECT tx.*, NOW() FROM tx;
```

<u>Predictions Section:</u>

What is the value of `value` for `id=4` in Step 2?
What is the value of `value` for `id=4` in Step 5? Is this the same as in Step 2?
What is the value of `value` for `id=4` in Step 7? Is this the same as in Step 2 and 5?

You should notice 2 things with Scenario E: Isolated Consistent Backup:

1. Terminal Window #1 has a transaction and changes *outside* this transaction will not affect any thing in this transaction. (for example, Terminal Window #2 changes don't affect our transaction in Terminal Window #1).

   This would be useful for creating an isolated consistent backup. Other users' changes could cause the database to be in an inconsistent state while we are backing up the database (if we weren't in a transaction).
2. The changes made outside the transaction in Terminal Window #1 are visible after the transaction has ended (either committed or rolled back).

Submit your screenshot of steps 1-7:

09_Scenario_E_consistent_backup.jpg

Submit

```
mysql> start transaction;
Query OK, 0 rows affected (0.00 sec)

mysql> select tx.*, now() from tx;
+----+-------+---------------------+
| id | value | now()               |
+----+-------+---------------------+
|  1 |  1000 |                     |
|  2 |  2000 |                     |
|  3 |  3000 |                     |
|  4 |  4000 |                     |
+----+-------+---------------------+
4 rows in set (0.00 sec)

mysql> select tx.*, now() from tx;
+----+-------+---------------------+
| id | value | now()               |
+----+-------+---------------------+
|  1 |  1000 |                     |
|  2 |  2000 |                     |
|  3 |  3000 |                     |
|  4 |  4000 |                     |
+----+-------+---------------------+
4 rows in set (0.00 sec)

mysql> rollback;
Query OK, 0 rows affected (0.00 sec)

mysql> select tx.*, now() from tx;
+----+-------+---------------------+
| id | value | now()               |
+----+-------+---------------------+
|  1 |  1000 |                     |
|  2 |  2000 |                     |
|  3 |  3000 |                     |
|  4 |  4994 |                     |
+----+-------+---------------------+
4 rows in set (0.00 sec)

mysql>
```

```
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql> update tx set value = 4994 where id = 4;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> select tx.*, now() from tx;
+----+-------+---------------------+
| id | value | now()               |
+----+-------+---------------------+
|  1 |  1000 |                     |
|  2 |  2000 |                     |
|  3 |  3000 |                     |
|  4 |  4994 |                     |
+----+-------+---------------------+
4 rows in set (0.00 sec)

mysql>
```

Step 7:

Scenario F: MyISAM is NOT Transaction Safe

Keep the 2 terminal windows logged in to MySQL as `root`.

IMPORTANT: Scroll the window up to the top so your screenshot only shows the current scenario.

Run the following commands on their appropriate Terminal Windows in sequence.

In this scenario we see what happens if we try to use a transaction with a MyISAM table.

	Terminal Window #1		Terminal Window #2
1	`START TRANSACTION;`		
2	`SELECT tx_myisam.*, NOW() FROM tx_myisam;`		
		3	`SELECT tx_myisam.*, NOW() FROM tx_myisam;`
4	`UPDATE tx_myisam SET value = 47 WHERE id = 4;`		
		5	`SELECT tx_myisam.*, NOW() FROM tx_myisam;`
6	`ROLLBACK;`		
7	`SELECT tx_myisam.*, NOW() FROM tx_myisam;`		

<u>Predictions Section:</u>

What is the value of `value` for `id=4` in Step 3?
What is the value of `value` for `id=4` in Step 5? Is this the same as in Step 3?
What is the value of `value` for `id=4` in Step 7?

You should notice the following with Scenario F: MyISAM is NOT Transaction Safe:

1. MyISAM is NOT transaction safe!
   Changes made inside a transaction take effect ***immediately***!
   MyISAM changes <u>cannot be rolled back</u>!

Submit your screenshot of steps 1-7 (after the rollback):

Submit

10_Scenario_F_myisam_is_not_safe.jpg

```
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql> start transaction;
Query OK, 0 rows affected (0.00 sec)

mysql> select tx_myisam.*, now() from tx_myisam;
+----+-------+---------------------+
| id | value | now()               |
+----+-------+---------------------+
|  1 |    10 | [illegible]         |
|  2 |    20 | [illegible]         |
|  3 |    30 | [illegible]         |
|  4 |    40 | [illegible]         |
+----+-------+---------------------+
4 rows in set (0.00 sec)

mysql> update tx_myisam set value = 47 where id = 4;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> rollback;
Query OK, 0 rows affected, 1 warning (0.00 sec)

mysql> select tx_myisam.*, now() from tx_myisam;
+----+-------+---------------------+
| id | value | now()               |
+----+-------+---------------------+
|  1 |    10 | [illegible]         |
|  2 |    20 | [illegible]         |
|  3 |    30 | [illegible]         |
|  4 |    47 | [illegible]         |
+----+-------+---------------------+
4 rows in set (0.00 sec)

mysql>
```

```
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql> select tx_myisam.*, now() from tx_myisam;
+----+-------+---------------------+
| id | value | now()               |
+----+-------+---------------------+
|  1 |    10 | [illegible]         |
|  2 |    20 | [illegible]         |
|  3 |    30 | [illegible]         |
|  4 |    40 | [illegible]         |
+----+-------+---------------------+
4 rows in set (0.00 sec)

mysql> select tx_myisam.*, now() from tx_myisam;
+----+-------+---------------------+
| id | value | now()               |
+----+-------+---------------------+
|  1 |    10 | [illegible]         |
|  2 |    20 | [illegible]         |
|  3 |    30 | [illegible]         |
|  4 |    47 | [illegible]         |
+----+-------+---------------------+
4 rows in set (0.00 sec)

mysql>
```

Step 8:

Scenario G: MyISAM Read Locks Block Writers

Keep the 2 terminal windows logged in to MySQL as `root`.

Run this command to reset the data back to the initial values (if needed):
`UPDATE tx_myisam SET value = id * 10;`

IMPORTANT: Scroll the window up to the top so your screenshot only shows the current scenario.

Run the following commands on their appropriate Terminal Windows in sequence.

In order to create a consistent backup in MyISAM, since we can't do transactions, we have to lock each and every table in the entire database. What effect does locking a table have other users?

	Terminal Window #1		Terminal Window #2
1	`LOCK TABLE tx_myisam READ;`		
2	`SELECT tx_myisam.*, NOW() FROM tx_myisam;`		
		3	`UPDATE tx_myisam SET value = 46 WHERE id = 4;`
4	`UNLOCK TABLES;`		
5	`SELECT tx_myisam.*, NOW() FROM tx_myisam;`		
		6	`SELECT tx_myisam.*, NOW() FROM tx_myisam;`

<u>Predictions Section:</u>

What happens when you try to run Step 3?
What happens when Step 4 completes?
What is the value of `value` for `id=4` in Step 5?
What is the value of `value` for `id=4` in Step 6?

You should notice 2 things with Scenario G: MyISAM Read Locks Block Writers:

1. The lock on the MyISAM table in Terminal Window #1 locks all users from writing to those tables for all other users (for example Terminal Window #2).
2. MyISAM writes are blocked by read locks and will wait (possibly a long time) until the locks are released with an `UNLOCK` command.

Submit your screenshot of steps 1-3 (before the unlock):

11_Scenario_G_before_unlock.jpg

Submit

```
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql> lock table tx_myisam read;
Query OK, 0 rows affected (0.02 sec)

mysql> select tx_myisam.*, now() from tx_myisam;
+----+-------+---------------------+
| id | value | now()               |
+----+-------+---------------------+
|  1 |    10 |                     |
|  2 |    20 |                     |
|  3 |    30 |                     |
|  4 |    40 |                     |
+----+-------+---------------------+
4 rows in set (0.00 sec)

mysql>
```

```
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
```

Step 9:

Scenario H: Deadlock

Keep the 2 terminal windows logged in to MySQL as `root`.

Run this command to reset the data back to the initial values (if needed):
`UPDATE tx SET value = id * 1000;`

IMPORTANT: Scroll the window up to the top so your screenshot only shows the current scenario.

Run the following commands on their appropriate Terminal Windows in sequence.

What happens when 2 transactions fight after the same data?

	Terminal Window #1		Terminal Window #2
1	`START TRANSACTION;`		
		2	`START TRANSACTION;`
3	`UPDATE tx SET value = 1743 WHERE id = 1;`		
		4	`UPDATE tx SET value = 2880 WHERE id = 2;`
		5	`UPDATE tx SET value = 1880 WHERE id = 1;`
6	`UPDATE tx SET value = 2743 WHERE id = 2;`		
7	`ROLLBACK;`		
		8	`ROLLBACK;`

<u>Predictions Section:</u>

Which step will create the deadlock?

<u>Note:</u> If you wait too long (>60 seconds) between running steps 5 and 6, you will likely get an error like:
`ERROR 1205 (HY000): Lock wait timeout exceeded; try restarting transaction`
This is NOT a deadlock! Rollback both terminal windows and try again.

You should notice 3 things with Scenario H: Deadlock:

1. At Step 5, Terminal Window #2 locks (hangs), waiting for Terminal Window #1 to either rollback or commit their transaction. <u>Note:</u> This is **not a deadlock** (at least not yet).
2. If Terminal Window #2 is waiting on a lock from Terminal Window #1 and Terminal Window #1 is waiting on a lock from Terminal Window #2, then **a deadlock occurs**.
3. MySQL before granting a lock on a resource, such as a row or table, checks all other locks and ensures that a deadlock won't be created. If granting a lock would create a deadlock, MySQL aborts a transaction to prevent the deadlock.

Submit your screenshot of steps 1-8:

13_Scenario_H_deadlock.jpg

Submit

```
mysql>
mysql>
mysql> start transaction;
Query OK, 0 rows affected (0.00 sec)

mysql> update tx set value = 1743 where id = 1;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> update tx set value = 2743 where id = 2;
ERROR [redacted] Deadlock found [redacted]

mysql> rollback;
Query OK, 0 rows affected (0.00 sec)

mysql>
```

```
mysql>
mysql> start transaction;
Query OK, 0 rows affected (0.00 sec)

mysql> update tx set value = 2880 where id = 2;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> update tx set value = 1880 where id = 1;
Query OK, 1 row affected (5.06 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> rollback;
Query OK, 0 rows affected (0.00 sec)

mysql>
```

Run the following command:

```
SHOW ENGINE INNODB STATUS;
```

Find the section labelled:

```
------------------------
LATEST DETECTED DEADLOCK
------------------------
```

Get a screenshot of all the content up to and including:

```
*** WE ROLL BACK TRANSACTION (2)
```

Use multiple screenshots if necessary:

Submit your screenshots:

14_Innodb_status_1.jpg

Submit

```
------------------------
LATEST DETECTED DEADLOCK
------------------------
2024-10-28 12:32:23 0x2018
*** (1) TRANSACTION:
TRANSACTION 27717, ACTIVE 8 sec starting index read
mysql tables in use 1, locked 1
LOCK WAIT 3 lock struct(s), heap size 1128, 2 row lock(s), undo log entries 1
MySQL thread id 28, OS thread handle 174380, query id 2473088 localhost ::1 root updating
update tx set value = 1880 where id = 1

*** (1) HOLDS THE LOCK(S):
RECORD LOCKS space id 252 page no 4 n bits 80 index PRIMARY of table `testing`.`tx` trx id 27717 lock_mode X locks rec but not gap
Record lock, heap no 3 PHYSICAL RECORD: n_fields 5; compact format; info bits 0
[redacted]

*** (1) WAITING FOR THIS LOCK TO BE GRANTED:
RECORD LOCKS space id 252 page no 4 n bits 80 index PRIMARY of table `testing`.`tx` trx id 27717 lock_mode X locks rec but not gap waiting
Record lock, heap no 2 PHYSICAL RECORD: n_fields 5; compact format; info bits 0
[redacted]
```

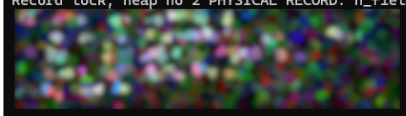
Submit your screenshots:

Submit

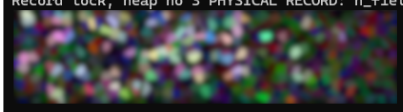
15_Innodb_status_2.jpg

```
*** (2) TRANSACTION:
TRANSACTION 27716, ACTIVE 16 sec starting index read
mysql tables in use 1, locked 1
LOCK WAIT 3 lock struct(s), heap size 1128, 2 row lock(s), undo log entries 1
MySQL thread id 29, OS thread handle 182612, query id 2473089 localhost ::1 root updating
update tx set value = 2743 where id = 2

*** (2) HOLDS THE LOCK(S):
RECORD LOCKS space id 252 page no 4 n bits 80 index PRIMARY of table `testing`.`tx` trx id 27716 lock_mode X locks rec but not gap
Record lock, heap no 2 PHYSICAL RECORD: n_fields 5; compact format; info bits 0
```



```
*** (2) WAITING FOR THIS LOCK TO BE GRANTED:
RECORD LOCKS space id 252 page no 4 n bits 80 index PRIMARY of table `testing`.`tx` trx id 27716 lock_mode X locks rec but not gap waiting
Record lock, heap no 3 PHYSICAL RECORD: n_fields 5; compact format; info bits 0
```



```
*** WE ROLL BACK TRANSACTION (2)
```

The command `SHOW ENGINE INNODB STATUS;` will provide us the most recent deadlock and a description of what caused it.

Notice there are 2 transactions conflicting in this deadlock – transaction 27716 and transaction 27717.

Transaction 27716 has a lock on: `update tx set value = 2880 where id = 2`
and is waiting on a lock to be granted on `id = 1`.
Transaction 27717 has a lock on: `update tx set value = 1743 where id = 1`
and is waiting on a lock to be granted on `id = 2`.

The 2 transactions are waiting for each other and can't proceed until the other releases their lock. MySQL will detect this deadlock and kill (rollback) one of the transactions to prevent a permanent deadlock.

In a real application, if you are experiencing deadlocks, you would try your transactions again at a later time if you get a deadlock error. Hopefully, at a later time, other transactions won't conflict with yours.

Step 10:

Scenario I: Implicit Commit with DDL

Keep the 2 terminal windows logged in to MySQL as `root`.

Run this command to reset the data back to the initial values (if needed):
`UPDATE tx SET value = id * 1000;`

IMPORTANT: Scroll the window up to the top so your screenshot only shows the current scenario.

Run the following commands on their appropriate Terminal Windows in sequence.

There are a few commands that you should be careful with when running commands inside a transaction.

Terminal Window #1

```
1  START TRANSACTION;
2  UPDATE tx SET value = 3518
   WHERE id = 3;
3  SELECT tx.*, NOW() FROM tx;

5  ALTER TABLE tx
   ADD COLUMN text VARCHAR(45)
   NULL AFTER value;

7  SELECT tx.*, NOW() FROM tx;
```

Terminal Window #2

```
4  SELECT tx.*, NOW() FROM tx;

6  SELECT tx.*, NOW() FROM tx;
```

<u>Predictions Section:</u>

What is the value of `value` for `id=3` in Step 3?
What is the value of `value` for `id=3` in Step 4?
What is the value of `value` for `id=3` in Step 6?

You should notice 2 things with Scenario I: Implicit Commit:

1. An `ALTER TABLE` command had the same effect as running a `COMMIT;` command first. This means our transaction will finalize and everything inside will be committed.
2. You cannot use Data Definition Language (DDL) commands inside a transaction and preserve the state of your transaction. DDL commands such as ALTER TABLE, CREATE DATABASE, CREATE TABLE, CREATE TRIGGER, CREATE ROLE, DROP DATABASE, DROP TABLE, DROP ROLE, TRUNCATE TABLE will all trigger an implicit commit and finalize the transaction. Avoid these commands while in a transaction. For a full list of implicit commit commands: https://dev.mysql.com/doc/refman/8.4/en/implicit-commit.html

Submit your screenshot of steps 1-7:

Submit

16_Scenario_I_implicit_commit.jpg

```
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql> start transaction;
Query OK, 0 rows affected (0.00 sec)

mysql> update tx set value = 3518 where id = 3;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> select tx.*, now() from tx;
+----+-------+---------------------+
| id | value | now()               |
+----+-------+---------------------+
|  1 |  1000 | [illegible]         |
|  2 |  2000 | [illegible]         |
|  3 |  3518 | [illegible]         |
|  4 |  4000 | [illegible]         |
+----+-------+---------------------+
4 rows in set (0.00 sec)

mysql> ALTER TABLE tx ADD COLUMN text VARCHAR(45) NULL AFTER value;
Query OK, 0 rows affected (0.06 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> select tx.*, now() from tx;
+----+-------+------+---------------------+
| id | value | text | now()               |
+----+-------+------+---------------------+
|  1 |  1000 | NULL | [illegible]         |
|  2 |  2000 | NULL | [illegible]         |
|  3 |  3518 | NULL | [illegible]         |
|  4 |  4000 | NULL | [illegible]         |
+----+-------+------+---------------------+
4 rows in set (0.00 sec)

mysql>
```

```
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql> select tx.*, now() from tx;
+----+-------+---------------------+
| id | value | now()               |
+----+-------+---------------------+
|  1 |  1000 | [illegible]         |
|  2 |  2000 | [illegible]         |
|  3 |  3000 | [illegible]         |
|  4 |  4000 | [illegible]         |
+----+-------+---------------------+
4 rows in set (0.00 sec)

mysql> select tx.*, now() from tx;
+----+-------+------+---------------------+
| id | value | text | now()               |
+----+-------+------+---------------------+
|  1 |  1000 | NULL | [illegible]         |
|  2 |  2000 | NULL | [illegible]         |
|  3 |  3518 | NULL | [illegible]         |
|  4 |  4000 | NULL | [illegible]         |
+----+-------+------+---------------------+
4 rows in set (0.00 sec)

mysql>
```

Step 11:

Scenario J: Uncommitted Transactions Prevent Other's DDL

Keep the 2 terminal windows logged in to MySQL as `root`.

Run this command to reset the data back to the initial values (if needed):
`UPDATE tx SET value = id * 1000;`

IMPORTANT: Scroll the window up to the top so your screenshot only shows the current scenario.

Run the following commands on their appropriate Terminal Windows in sequence.

In this scenario we will see that uncommitted transactions create locks that can block other transactions.

Terminal Window #1

```
1  START TRANSACTION;
2  UPDATE tx SET value = 3941
   WHERE id = 3;
3  SELECT tx.*, NOW() FROM tx;


5  ROLLBACK;
```

Terminal Window #2

```
4  ALTER TABLE tx
   DROP COLUMN text;
```

<u>Predictions Section:</u>

What happens when you try to run Step 4?
What happens when Step 5 completes?

You should notice with Scenario J: Uncommitted Transactions Prevent Other's DDL:

1. Starting a transaction and updating a row in a table puts a lock on that row. That lock remains until the transaction ends with either a ROLLBACK or a COMMIT.
   DDL commands such as an `ALTER TABLE` command cannot complete while there is a lock on the any of the rows on the table (or on the table itself).

Submit your screenshot of steps 1-5:

Submit

17_Scenario_J_uncommitted_locks.jpg

```
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql> start transaction;
Query OK, 0 rows affected (0.00 sec)

mysql> update tx set value = 3941 where id = 3;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> select tx.*, now() from tx;
+----+-------+------+---------------------+
| id | value | text | now()               |
+----+-------+------+---------------------+
|  1 |  1000 | NULL |                     |
|  2 |  2000 | NULL |                     |
|  3 |  3941 | NULL |                     |
|  4 |  4000 | NULL |                     |
+----+-------+------+---------------------+
4 rows in set (0.00 sec)

mysql> rollback;
Query OK, 0 rows affected (0.00 sec)

mysql>
```

```
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
```

Step 12:

Scenario K: Isolation Level – Read Uncommitted

Keep the 2 terminal windows logged in to MySQL as `root`.

Run this command to reset the data back to the initial values (if needed):
`UPDATE tx SET value = id * 1000;`

IMPORTANT: Scroll the window up to the top so your screenshot only shows the current scenario.

Run the following commands on their appropriate Terminal Windows in sequence.

There are 4 levels of isolation in MySQL:

1. READ UNCOMMITTED (Fewer locks [more concurrency], less isolation [more dirty reads])
2. READ COMMITTED
3. REPEATABLE READ *(MySQL InnoDB's default isolation level)*
4. SERIALIZABLE (More locks [less concurrency], more isolation [less dirty reads])

What happens when we change the isolation level to READ UNCOMMITTED?

Terminal Window #1

```
2  START TRANSACTION;
3  SELECT tx.*, NOW() FROM tx;
4  UPDATE tx SET value = 2291
   WHERE id = 2;

6  ROLLBACK;
7  SELECT tx.*, NOW() FROM tx;
```

Terminal Window #2

```
1  SET SESSION TRANSACTION
   ISOLATION LEVEL
   READ UNCOMMITTED;

5  SELECT tx.*, NOW() FROM tx;

8  SELECT tx.*, NOW() FROM tx;
```

<u>Predictions Section:</u>

What is the value of `value` for `id=2` in Step 3?
What is the value of `value` for `id=2` in Step 5? Is this the same as in Step 3?
What is the value of `value` for `id=2` in Step 7?
What is the value of `value` for `id=2` in Step 8? Is this the same as in Step 5?

You should notice 2 things with Scenario K: Isolation Level – Read Uncommitted:

1. Comparing Scenario K to Scenario A, we will notice now that Terminal Window #2, because it has Isolation Level READ UNCOMMITTED, it does just that, it reads the uncommitted data from the transaction in Terminal Window #1.
2. Scenario A, in contrast, used the default Isolation Level - REPEATABLE READ, so Terminal Window #2 was NOT able to see the uncommitted changes.

Submit your screenshot of steps 1-8:

18_Scenario_K_read_uncommitted.jpg

Submit

```
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql> start transaction;
Query OK, 0 rows affected (0.00 sec)

mysql> select tx.*, now() from tx;
+----+-------+---------------------+
| id | value | now()               |
+----+-------+---------------------+
|  1 |  1000 |                     |
|  2 |  2000 |                     |
|  3 |  3000 |                     |
|  4 |  4000 |                     |
+----+-------+---------------------+
4 rows in set (0.00 sec)

mysql> update tx set value = 2291 where id = 2;
Query OK, 1 row affected (0.02 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> rollback;
Query OK, 0 rows affected (0.00 sec)

mysql> select tx.*, now() from tx;
+----+-------+---------------------+
| id | value | now()               |
+----+-------+---------------------+
|  1 |  1000 |                     |
|  2 |  2000 |                     |
|  3 |  3000 |                     |
|  4 |  4000 |                     |
+----+-------+---------------------+
4 rows in set (0.00 sec)

mysql>
```

```
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql> set session transaction isolation level read uncommitted;
Query OK, 0 rows affected (0.00 sec)

mysql> select tx.*, now() from tx;
+----+-------+---------------------+
| id | value | now()               |
+----+-------+---------------------+
|  1 |  1000 |                     |
|  2 |  2291 |                     |
|  3 |  3000 |                     |
|  4 |  4000 |                     |
+----+-------+---------------------+
4 rows in set (0.00 sec)

mysql> select tx.*, now() from tx;
+----+-------+---------------------+
| id | value | now()               |
+----+-------+---------------------+
|  1 |  1000 |                     |
|  2 |  2000 |                     |
|  3 |  3000 |                     |
|  4 |  4000 |                     |
+----+-------+---------------------+
4 rows in set (0.00 sec)

mysql>
```

Step 13:

Scenario L: Isolation Level – Serializable

Keep the 2 terminal windows logged in to MySQL as `root`.

Run this command to reset the data back to the initial values (if needed):
`UPDATE tx SET value = id * 1000;`

IMPORTANT: Scroll the window up to the top so your screenshot only shows the current scenario.

Run the following commands on their appropriate Terminal Windows in sequence.

There are 4 levels of isolation in MySQL:

1. READ UNCOMMITTED (Fewer locks [more concurrency], less isolation [more dirty reads])
2. READ COMMITTED
3. REPEATABLE READ *(MySQL InnoDB's default isolation level)*
4. SERIALIZABLE (More locks [less concurrency], more isolation [less dirty reads])

What happens when we change the isolation level to SERIALIZABLE?

Terminal Window #1

```
1  SET SESSION TRANSACTION
   ISOLATION LEVEL SERIALIZABLE;
3  START TRANSACTION;

5  SELECT tx.*, NOW() FROM tx
   WHERE id = 2;



8  ROLLBACK;
```

Terminal Window #2

```
2  SET SESSION TRANSACTION
   ISOLATION LEVEL REPEATABLE
   READ;
4  START TRANSACTION;

6  UPDATE tx SET value = 1006
   WHERE id = 1;
7  UPDATE tx SET value = 2997
   WHERE id = 2;

9  ROLLBACK;
```

<u>Predictions Section:</u>

What happens when you try to run Step 6?
What happens when you try to run Step 7?
What happens when Step 8 completes?

You should notice with Scenario L: Isolation Level – Serializable:

1. Comparing Scenario L to Scenario K (and Scenario A), we will notice now that Terminal #2 blocks when trying to update a value that was read by Terminal #1. SERIALIZABLE is the highest level of isolation and requires that every value that is read (via a SELECT) cannot be changed during that transaction. Serialized reads block writes for those rows (in this case id=2).

Submit your screenshot of steps 1-9:

19_Scenario_L_serialization.jpg

Submit

```
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql> set session transaction isolation level serializable;
Query OK, 0 rows affected (0.00 sec)

mysql> start transaction;
Query OK, 0 rows affected (0.00 sec)

mysql> select tx.*, now() from tx where id = 2;
+----+-------+---------------------+
| id | value | now()               |
+----+-------+---------------------+
|  2 |  2000 |[illegible]|
+----+-------+---------------------+
1 row in set (0.00 sec)

mysql> rollback;
Query OK, 0 rows affected (0.00 sec)

mysql>
```

```
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql>
mysql> set session transaction isolation level repeatable read;
Query OK, 0 rows affected (0.00 sec)

mysql> start transaction;
Query OK, 0 rows affected (0.00 sec)

mysql> update tx set value = 1006 where id = 1;
Query OK, 1 row affected (0.02 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> update tx set value = 2997 where id = 2;
Query OK, 1 row affected (4.85 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> rollback;
Query OK, 0 rows affected (0.02 sec)

mysql>
```